

1조

내 주변 맛집 기록 어플 한 입만!

강연주 김서임 박은호 윤준영



목차

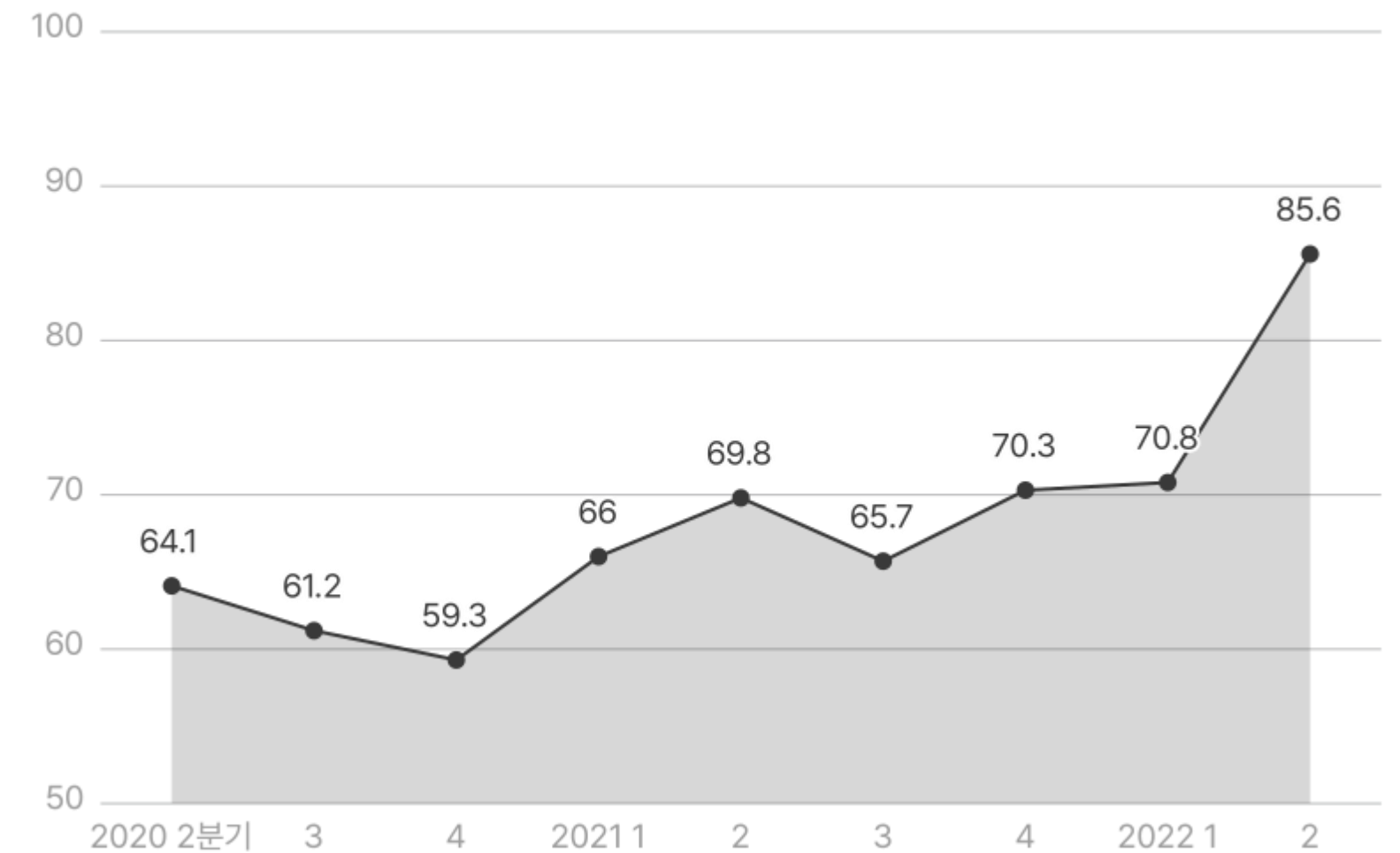
1. 프로젝트 소개
2. 프로젝트 팀원 소개 및 역할
3. 프로젝트 개발 일정
4. 개발 방식 및 협업 방식
5. 프로젝트 기획
6. 프로젝트 개발
7. 프로젝트 결과 및 시연

기획 의도

현대인의 급변하는 식생활??

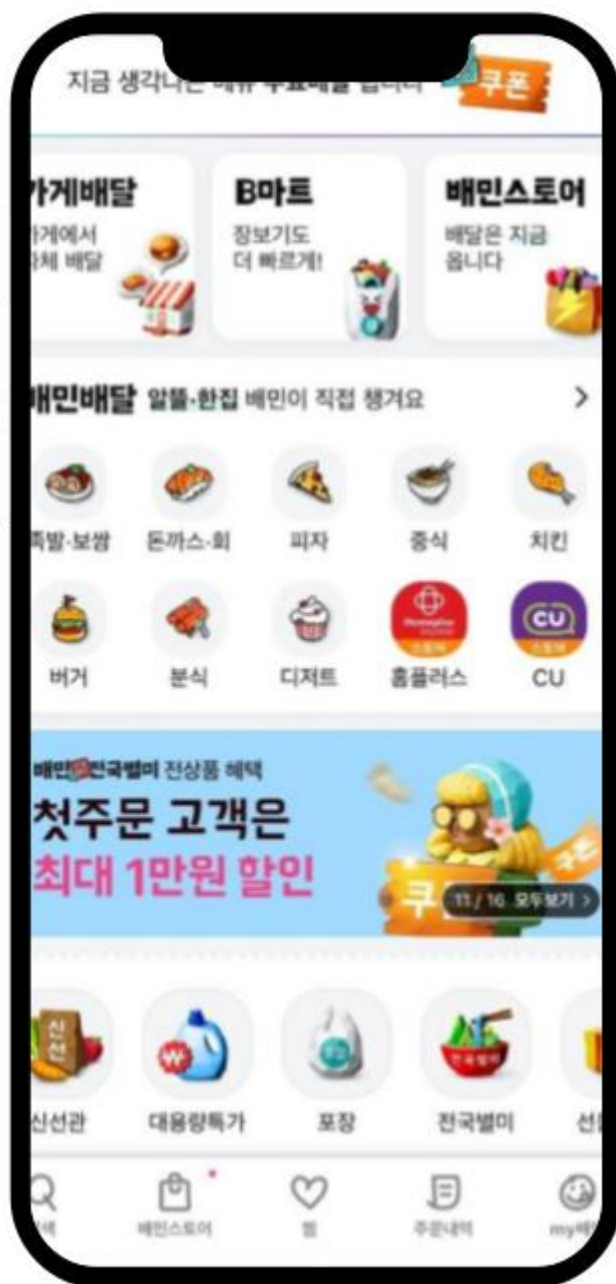


외식산업 추이



유사앱 비교

배달의 민족



- 위치 기반 맛집 탐색 및 배달 서비스 제공
- 사용자 리뷰 및 별점 평가
- 배달 및 포장 서비스 지원

인스타그램



- 사진 및 영상 기반 소셜 네트워크
- 해시태그 및 위치 기반 검색 기능 제공
- 게시물 좋아요 및 댓글 기능

1조

팀원 소개 및 역할



back-end



김서임

- 일기 **CRUD** 및 전체**API** 개발
- 식당&일기 연동 API 개발
- 카카오 로그인 및 마이페이지 API연동
- PPT 기획 및 발표

back-end



박은호

- 식당 & 지도 API 개발 및 DB
- AWS EC2, EDS 서버 배포
- Spring Boot + MySQL 백엔드 구성

front-end



윤준영

- B파트 UI/UX 흐름 구성 및 구현
- 일기데이터 Retrofit API 연동
- 안드로이드 화면 통합
- 앱 서버 연동 로직 구현

front-end

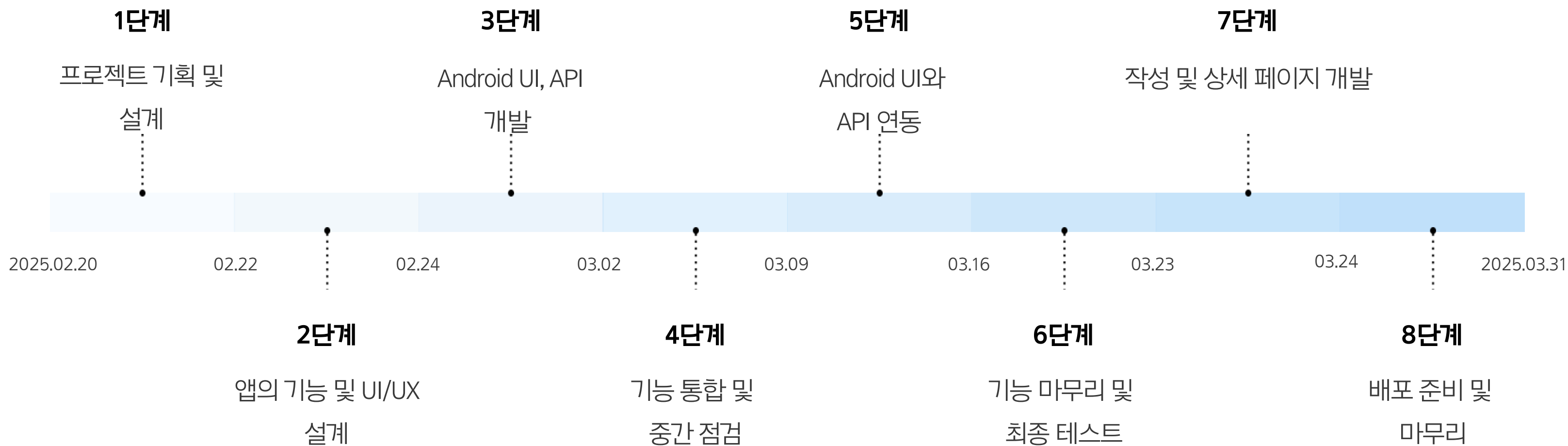


강연주

- A파트 UI/UX 흐름 구성 및 구현
- 식당데이터 Retrofit API 연동
- Figma를 활용한 앱 디자인
- ppt 기획 및 작성

개발 일정

Development Schedule



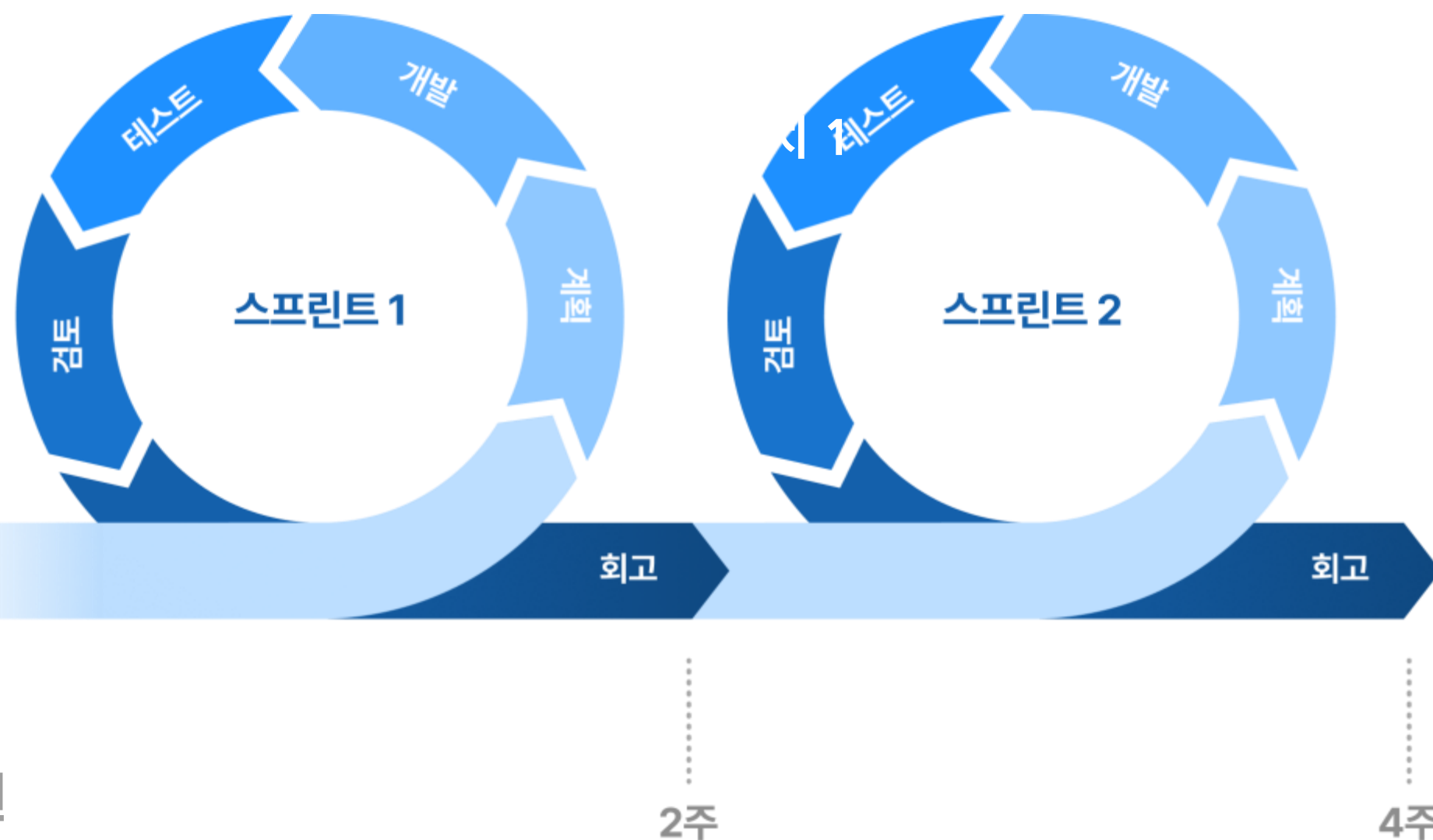
개발 방식

개발 방식?

애자일 개발 스크럼 방식⁺

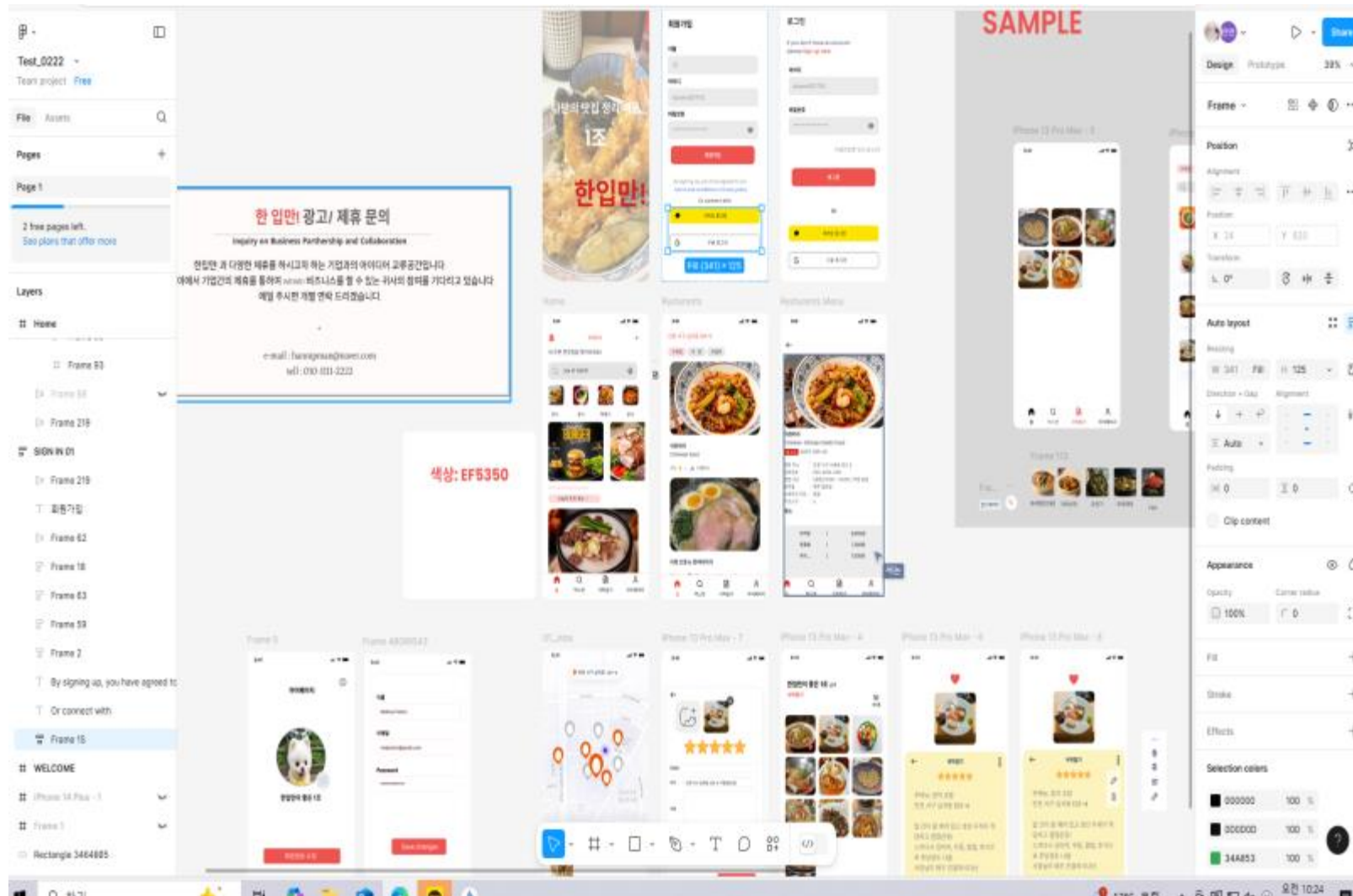
#애자일 스크럼 개발

1. 짧은 개발 주기(스프린트)로 빠른 피드백 반영
2. 유연한 변화 대응 & 체계적인 개발 프로세스
3. 팀 중심의 협업 & 지속적인 커뮤니케이션
4. 스프린트 회고(Retrospective)로 지속적인 개선



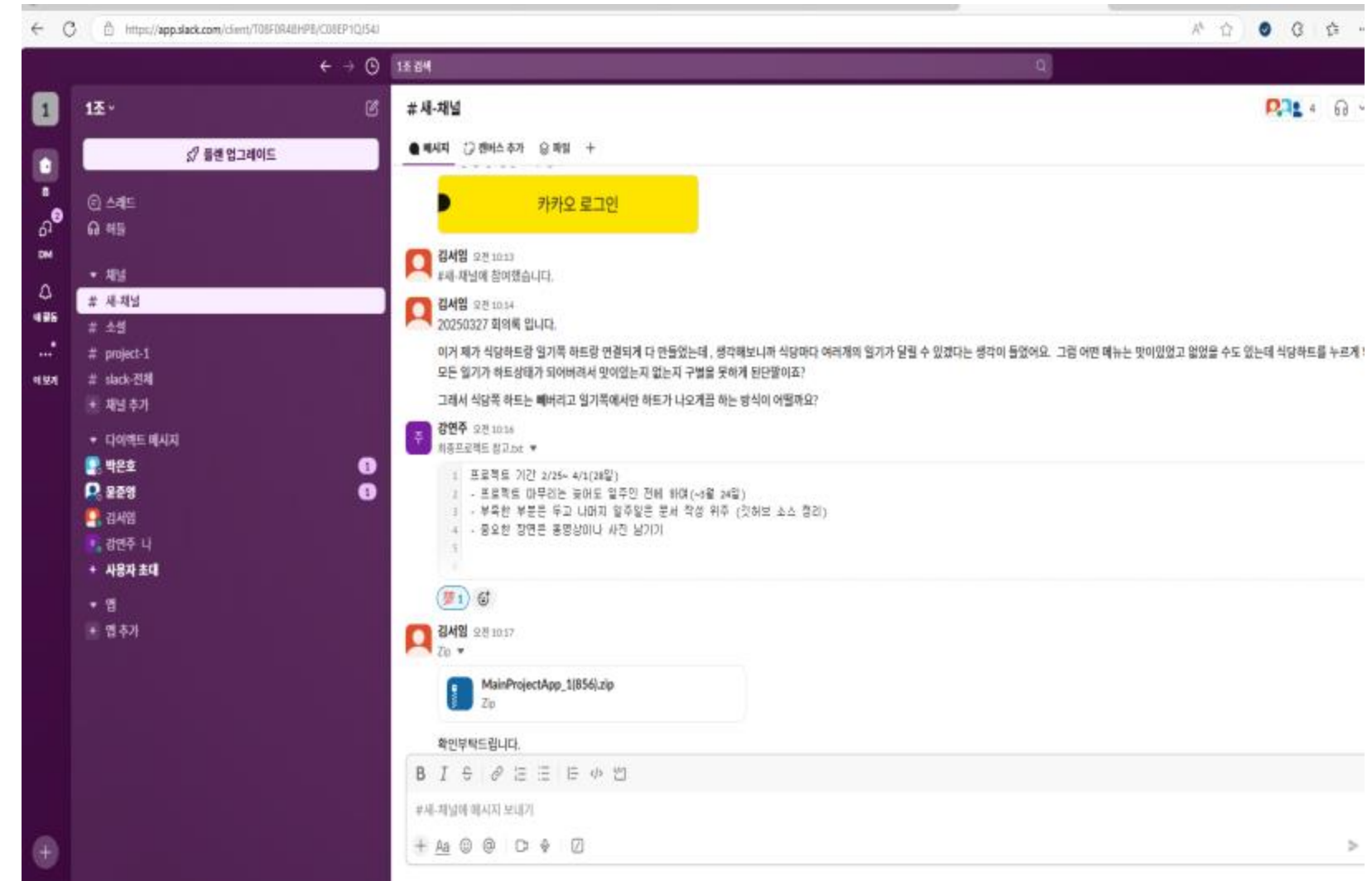
협업 방식

Figma



디자이너와 개발자가 함께 실시간으로 협업할 수 있는
클라우드 기반 UI/UX 디자인 및 프로토타이핑 툴

Slack



팀원들이 채널을 통해 실시간으로 소통하고 파일을
공유하며 협업할 수 있는 업무용 메시징 플랫폼

개발 기술

BACK-END

- Spring Boot Java 기반의 경량 웹 프레임워크, RESTful API 서버 구현에 활용 MySQL RDBMS로 데이터 영속성 확률. 식당, 사용자, 일기 등 다양한
- 도메인 데이터를 테이블로 관리. JPA를 활용한 ORM 매핑으로 케이드 클래스 기반 개발 지원.
- Spring Data JPA 반복적인 SQL 작성 없이 연결명을 기반으로 객체적 데이터
- 접근 구현. 식당-일기 가정 간개 설정, 다중 조건 조회 등 쉽게 처리.

FRONT-END

- Android Studio (Java)
전체 앱 화면 개발에 사용. 홈, 지도, 피드, 마이페이지 등 다양한 UI를 Fragment 기반으로 구성.
- Retrofit2
서버와 RESTful 방식으로 통신하는 HTTP 클라이언트.GET, POST, PUT, DELETE 등 다양한 API 요청을 간개하게 구현.
- Glide
서버에서 받은 이미지 URL을 바탕으로 빠른 로딩 및 캐시내시에 사용.
Feed, 상세 화면, 지도 바트시트 등 이미지가 필요한 UI에 사용.

개발 기술

DevOps

-Amazon EC2

백엔드 서버를 배포하기 위해 사용된 AWS 인스턴스.
.jar 파일 업로드 및 실행을 통해 앱 서비스 운영 환경 구축

-AWS RDS (MySQL)

EC2에서 분리된 DB 서버로 활용.
로컬에서 개발한 데이터 구조를 그대로 클라우드에서 운영

-FileZilla / SCP

EC2 서버에 백엔드 빌드 파일 및 설정 파일을 전송할 때 사용.

Integration & Connectivity

-Postman

API 연동 테스트 도구로 활용 요청/응답 구조 확인 및 오류 디버깅을 통해 개발 효율성을 향상

-Kakao Login API

OAuth 2.0 기반의 소셜 로그인 기능 구현
사용자 프로필 정보와 연동하여 마이페이지 구성에 활용

-Google Maps API

사용자의 현재 위치 기반 식당 탐색 및 마커 기반 지도 인터랙션 구현



프로젝트 개요



Location-based

위치 기반 직관적인
맛집 탐색 및 추천 기능



Food Diary

맞춤형 미식 기록 및
별점 기능



Dining Info

다양한 맛집 상세정보 제공

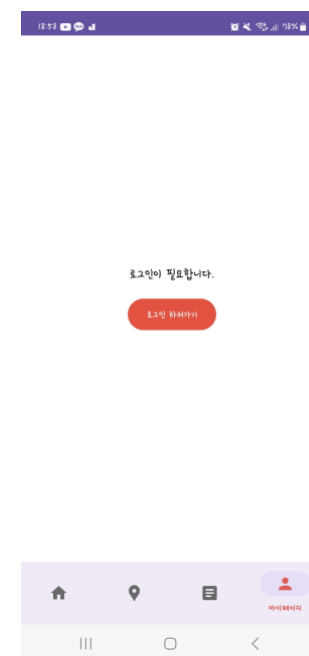
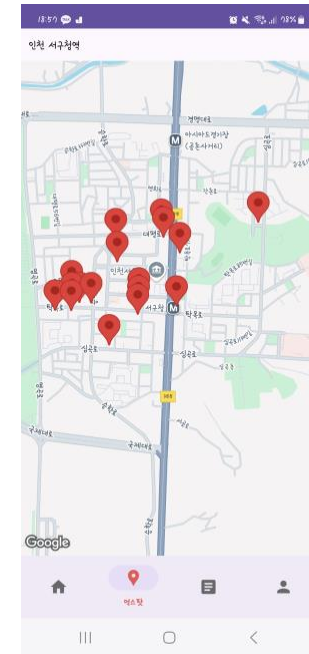
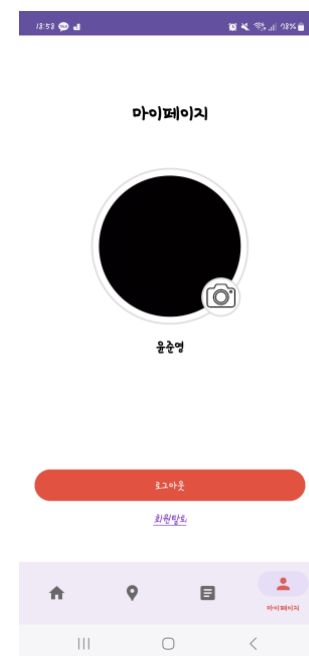
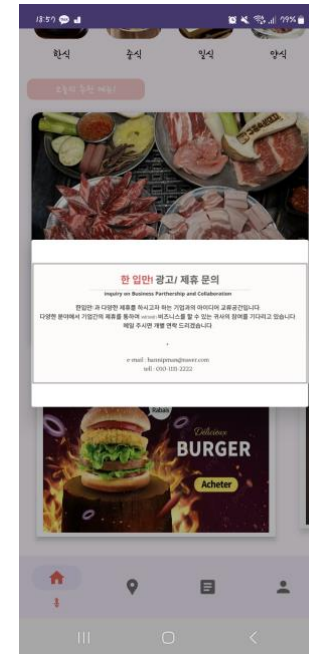
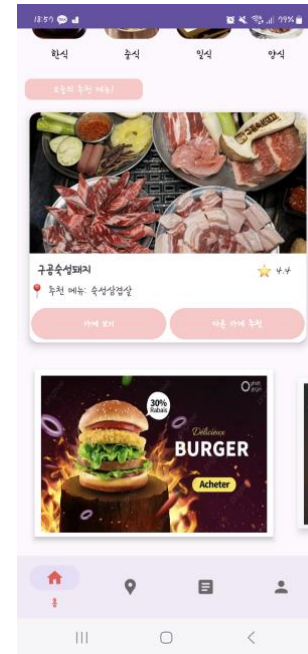
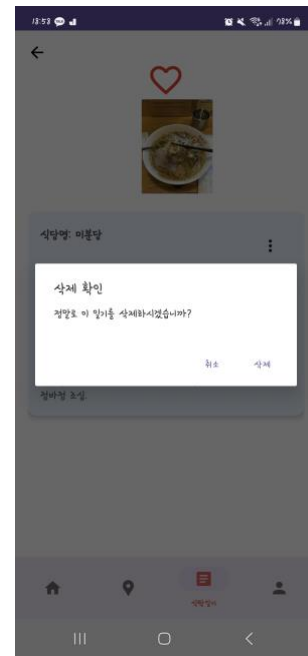
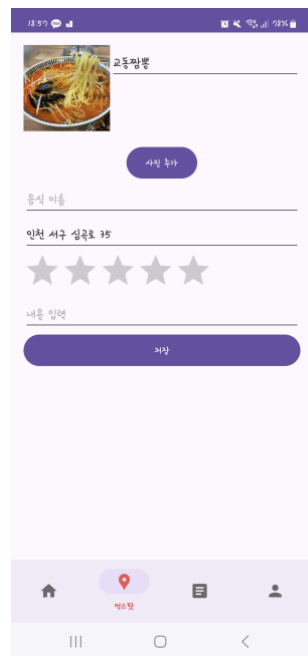
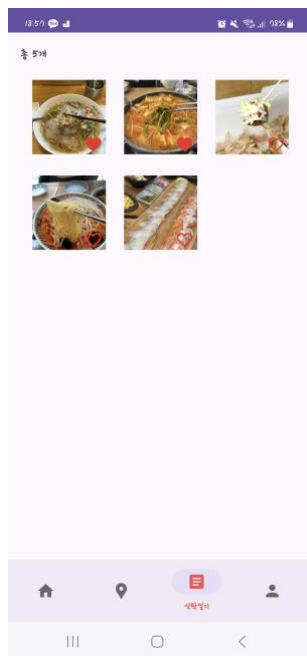
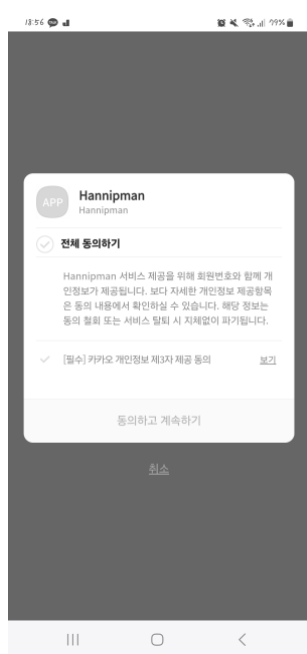
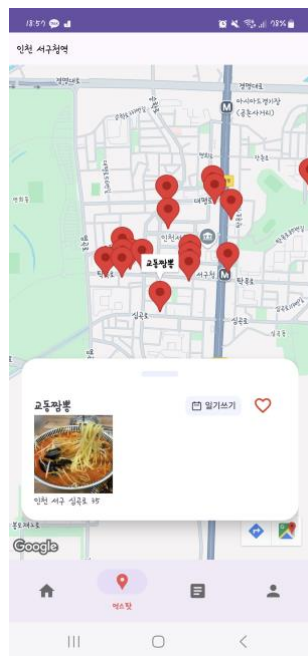


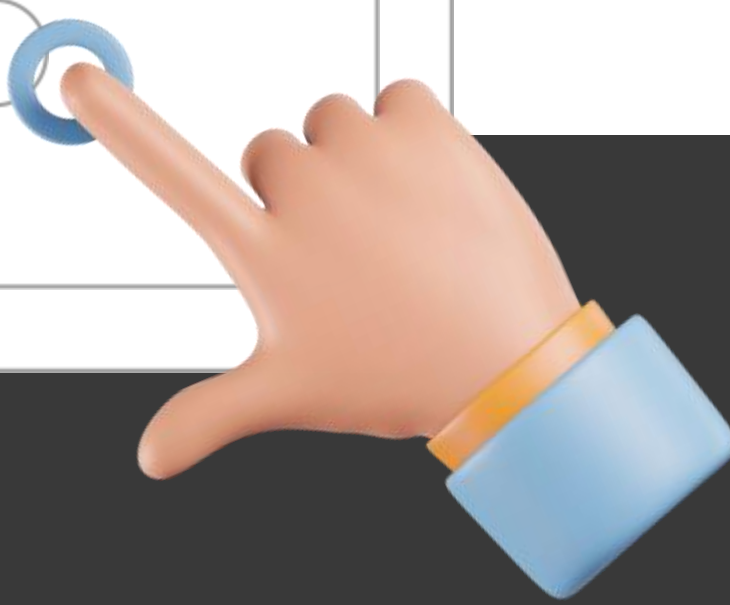
User Interface

간편한 사용자 인터페이스

화면 기획서

[Figma.com](https://www.figma.com)





한입만

Back-end

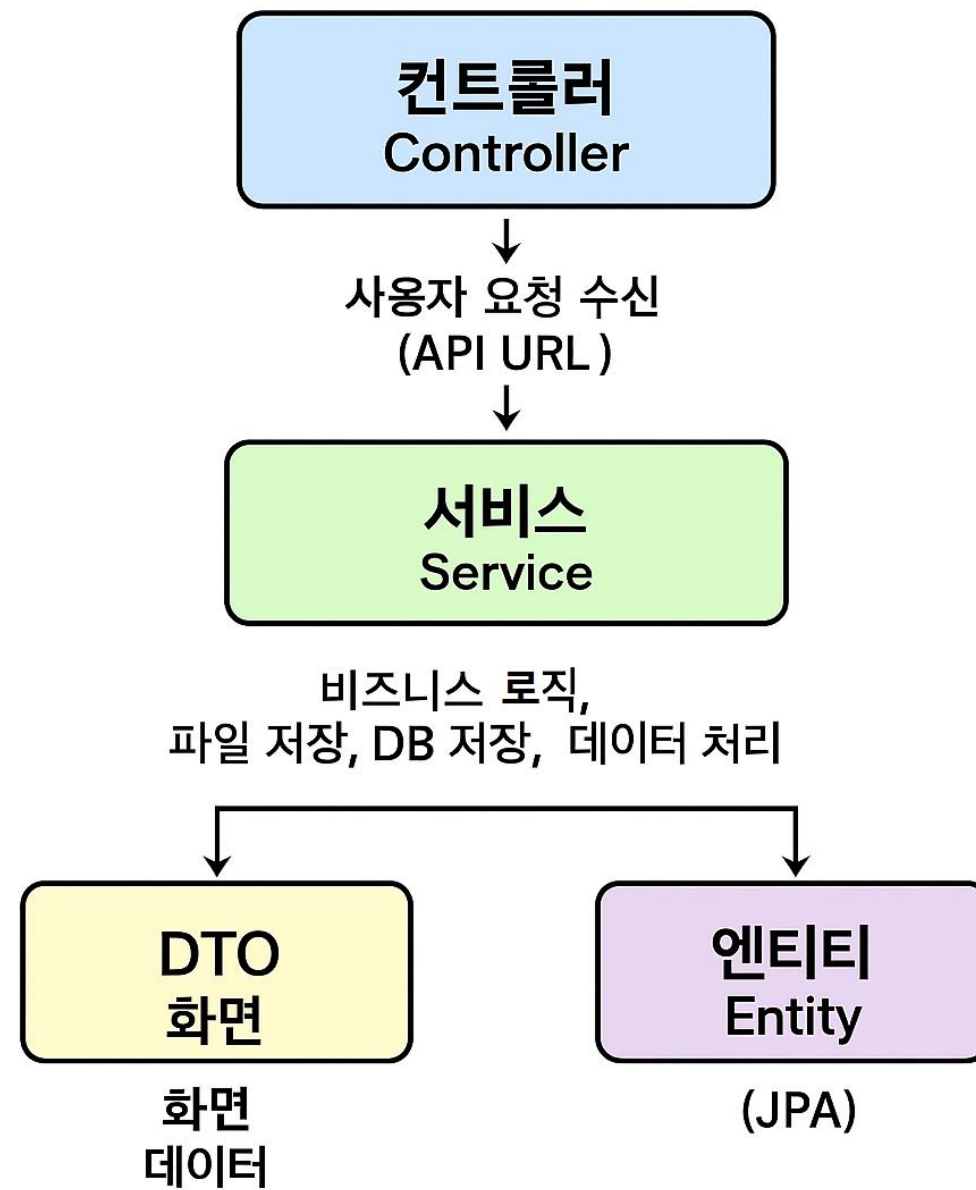
주요기능개발

#Spring Boot

#MySQL

#Spring Data JPA

Architecture # 백엔드 구조 및 흐름 개요



전체 흐름

요청 → 컨트롤러 → 서비스 → 리포지토리
→ 응답 DTO 반환

 사용자 요청 수신 → Controller

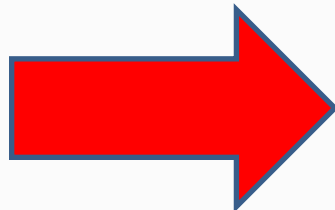
 핵심 로직 처리 → Service

 응답 데이터 전달 → DTO

DB 생성

```
@PostConstruct 0개의 사용위치
public void loadDataFromCSVAndImages() {
    String csvFilePath = "src/main/resources/data/restaurantDB.csv";
    String imageDirectoryPath = "src/main/resources/images/";

    Restaurant restaurant = new Restaurant();
    restaurant.setHeart(line[1].equals("1"));
    restaurant.setDiary(line[2].equals("1"));
    restaurant.setName(line[3]);
    restaurant.setMenu(Arrays.asList(line[4].split("regex: ","")));
    restaurant.setPrice(parsePrices(line[5]));
    restaurant.setRating(Double.parseDouble(line[6]));
    restaurant.setKategorie(line[7]);
    restaurant.setPhoneNumber(line[8]);
    restaurant.setBusinessHours(line[9]);
    restaurant.setClosedDays(line[10]);
    restaurant.setBreakTime(line[11]);
    restaurant.setKiosk(line[12]);
    restaurant.setPlace(line[13]);
    restaurant.setLatitude(Double.parseDouble(line[14]));
    restaurant.setLongitude(Double.parseDouble(line[15]));
}
```

[illegible]

CSV파일로 작성 CSV파일을 읽어 MySQL DB에 저장하는

```

1 1,1,1,조금조밥, "블루조밥, 생면어초밥, 모듬초밥, 민물장어초밥, 한우초밥, 초새우초밥, 연어불초밥", "19000,18000,18000,19000,19000,16000,19000"
2 2,0,0,국수궁전, "잔치국수,비빔국수,쫄면,냉국수,비빔밥,부추전", "6000,7000,7000,7000,7000,7000", 4.5,한식,0507-1349-9569,08:00-22:00
3 3,0,0,감자탕마을, "감자탕(소),등뼈해물찜(소),삼계탕,뼈해장국,생고기김치찌개,양훈비빔밥,냉면", "35000,45000,14000,10000,10000,10000,10000"
4 4,0,0,성원닭갈비, "닭갈비(소),닭갈비(중),닭갈비(대),치즈볶음밥,알밥,라면 떡사리,쫄면 우동사리", "29000,36000,44000,4000,4000,2000,2000"
5 5,1,1,교동짬뽕, "교동짬뽕,찰쌀탕수육 미니,삼선짬뽕,짜장면,쟁반짜장,우동", "9500,14000,13000,7000,9500,9500", 4.4,중식,0507-1442-8899,11:00-22:00
6 6,0,0,춘작곡, "소 와 갈비, 하동 선화꽃살 춘작 돼지고기국미 와 갈비탕(2대), 하동 된장찌개, 전골, 한호냉면", "48000,62000,19000,17000,10000"

```

DB 생성 일기 테이블 MySQL 저장

```

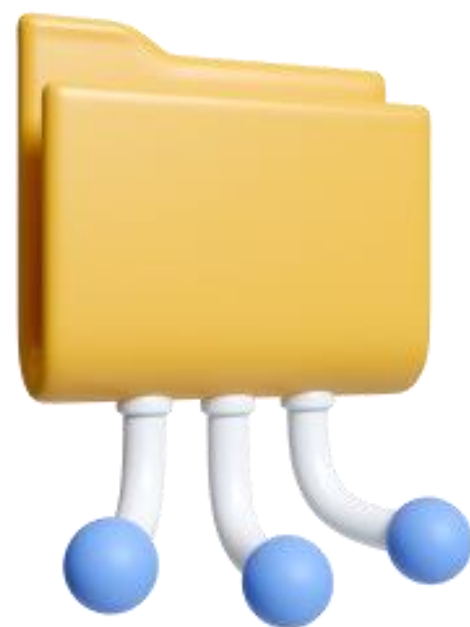
16 public class FoodDiary {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private Long id;
21
22     //연관된 식당 정보 (ManyToOne 관계)
23     @ManyToOne
24     @JoinColumn(name = "restaurant_id", nullable = false)
25     private Restaurant restaurant;
26
27     //유저가 작성한 일기 내용
28     @Column(nullable = false)
29     private String diaryText;
30
31     // 업로드된 사진 경로
32     @Lob
33     private String photoPath;
34
35     //생성 시간 (자동 생성, 수정 불가)
36     @Column(nullable = false, updatable = false)
37     @Temporal(TemporalType.TIMESTAMP)
38     private Date createdAt;
39     //유저가 직접 고른 별점
40     private Float rating;
41     // 유저가 직접 입력한 메뉴 이름
42     private String menuName;
43 }

```

일기 작성 시 Food_diary 테이블에 저장

[illegible]

API 구조 #식당 API #일기 API



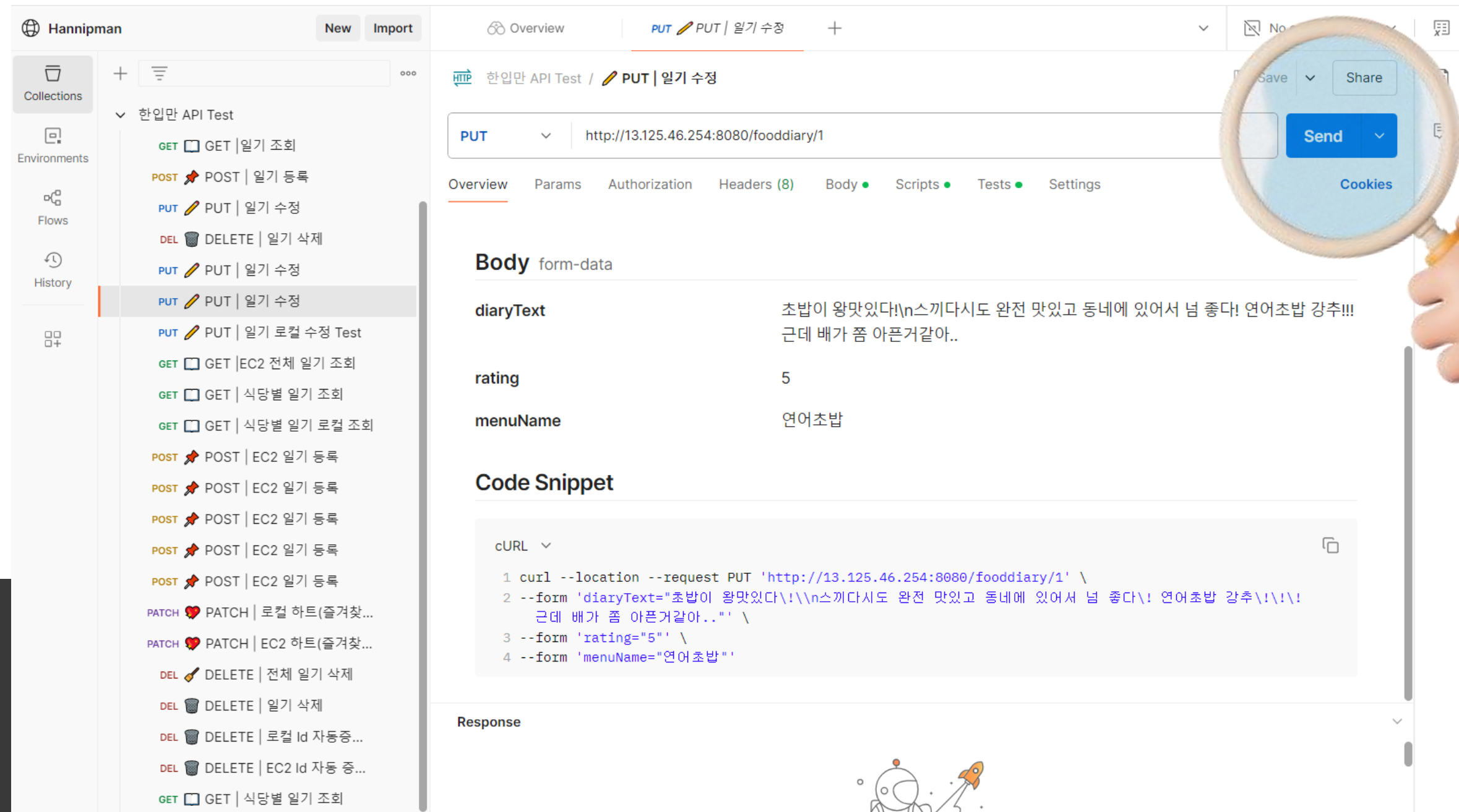
식당

전체 식당 조회하기(HTML)
특정 식당 이미지 가져오기
전체 식당 정보 가져오기
특정 식당정보 가져오기

일기

전체 일기 리스트 보기
일기 상세 페이지 보기
특정 식당의 일기 가져오기
일기 CRUD 기능 구현하기
일기 총 개수 세기

API 명세서 #Postman #JSON Hannipman Food Diary App



Hannipman New Import

Overview PUT PUT | 일기 수정

한입만 API Test / PUT | 일기 수정

PUT http://13.125.46.254:8080/fooddiary/1

Overview Params Authorization Headers (8) Body Scripts Tests Settings

Body form-data

diaryText	초밥이 왕맛있다!\n스끼다시도 완전 맛있고 동네에 있어서 넘 좋다! 연어초밥 강추!!! 근데 배가 좀 아픈거같아..
rating	5
menuName	연어초밥

Code Snippet

cURL

```
1 curl --location --request PUT 'http://13.125.46.254:8080/fooddiary/1' \  
2 --form 'diaryText="초밥이 왕맛있다!\n스끼다시도 완전 맛있고 동네에 있어서 넘 좋다! 연어초밥 강추!!!  
근데 배가 좀 아픈거같아.."' \  
3 --form 'rating="5"' \  
4 --form 'menuName="연어초밥"'
```

Response

AWS 업로드 **#EC2** **#RDS**

[illegible]

EC2 서버에 API 업로드

[Browse Documentation](#)

MySQL Connections

RDS 서버에 MySQL 연결

DB 테이블

식당정보 DB

일기 DB

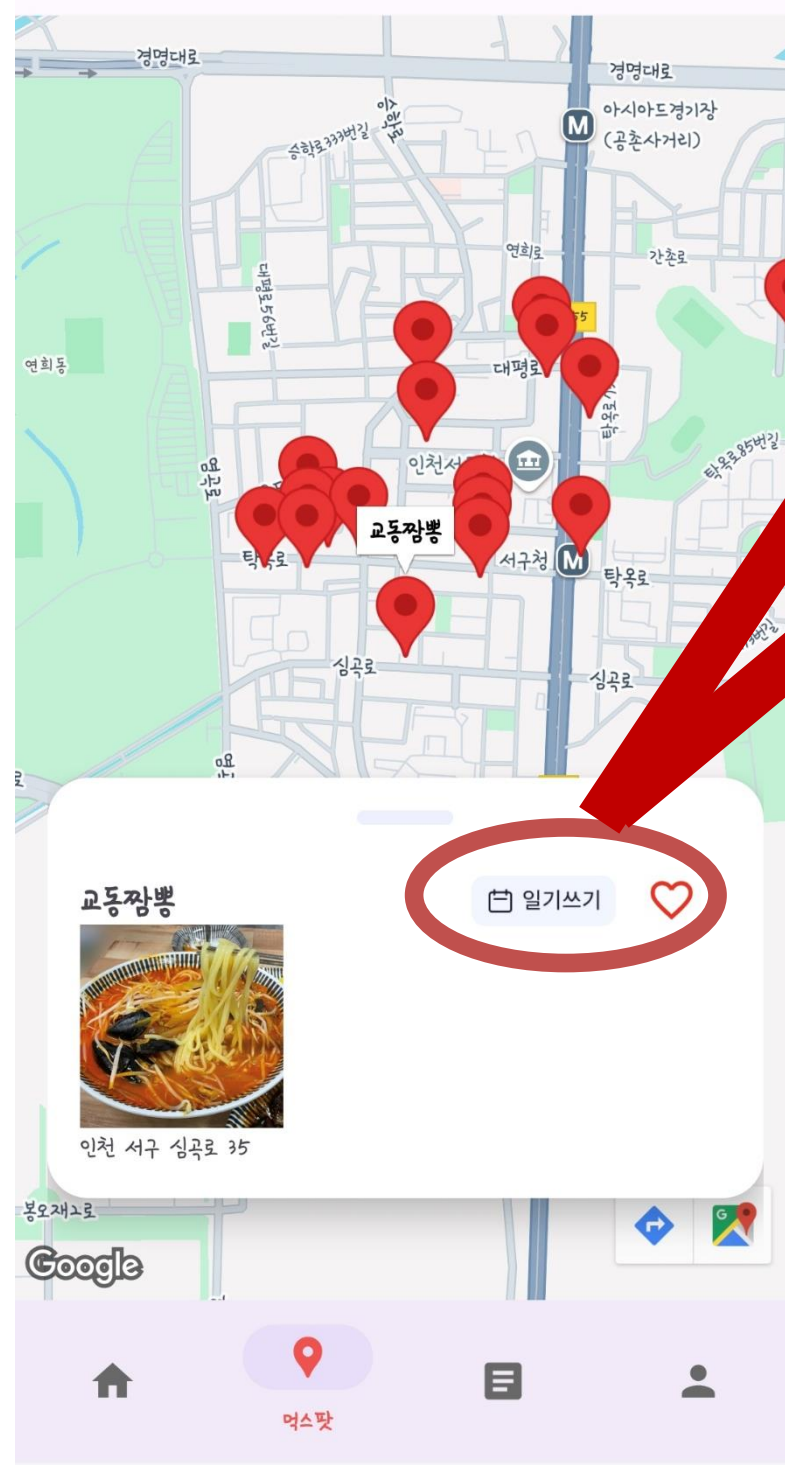
	id	break_time	business_hours	closed_days	diary	heart	image	kategorie	kiosk	latitude	lo
	18	없음	12:00~22:00	없음	0	0	BLOB	한식	없음	37.5294864043422	12
	19	없음	11:00~22:30	없음	0	0	BLOB	한식	없음	37.533887048764	12
	20	없음	11:00~21:00	없음	1	1	BLOB	중식	없음	37.5346639378439	12
	21	없음	08:00~22:00	없음	0	0	BLOB	양식	없음	37.5437286275368	12
	22	없음	11:00~21:00	월요일	0	0	BLOB	양식	있음	37.5442217804822	12
	23	없음	11:00~22:00	없음	0	0	BLOB	양식	없음	37.543890871844	12
	24	15:00~17:00	11:30~22:00	없음	0	0	BLOB	양식	있음	37.5338603917033	12
	25	없음	08:00~22:00	없음	0	0	BLOB	양식	있음	37.5227614575062	12
	26	15:00~17:00	11:00~21:00	수요일	0	0	BLOB	중식	있음	37.5470039273843	12
	27	없음	11:00~21:00	없음	0	0	BLOB	중식	있음	37.5463926279011	12
	28	없음	09:30~21:00	없음	0	0	BLOB	중식	없음	37.5479910532792	12
	29	14:00~17:00	11:30~04:00	없음	0	0	BLOB	일식	없음	37.5444239179483	12
	30	없음	10:00~21:00	토요일	1	1	BLOB	일식	없음	37.5447424631365	12

	id	created_at	diary_text	menu_name	photo_path
	1	2025-03-28 12:21:29.392...	초밥이 왕맛있다! 뽕스끼다시...	연어초밥	/uploads/restaurant_1/1.jpg
	2	2025-03-28 12:24:50.570...	고동짬뽕.. 여기 진짜 서구청 ...	짬뽕	/uploads/restaurant_5/2.jpg
	3	2025-03-28 12:25:58.925...	타코타코 타코로와~ 문어아저...	타코야끼	/uploads/restaurant_10/3.jpg
	4	2025-03-28 12:33:30.929...	고북샤브샤브 여기는 내 샤브 ...	고북샤브세트	/uploads/restaurant_15/4.jpg
	5	2025-03-28 12:34:31.780...	미분당은 혼자먹기좋은 혼밥집...	양지찜국수	/uploads/restaurant_20/5.jpg
	6	2025-03-28 12:36:04.149...	어푸키친 파스타가 야무짐. 투...	투움바 파스타	/uploads/restaurant_22/6.jpg
	7	2025-03-28 12:37:16.817...	라쿤카츠는 라쿤이 사장님이다...	등심카츠	/uploads/restaurant_30/7.jpg
	NULL	NULL	NULL	NULL	NULL

Restaurant

Food_diary

일기 API # 특정식당 일기 연동



교동짬뽕

사진 추가

음식 이름

인천 서구 심곡로 35

★ ★ ★ ★ ★

내용 입력

저장



사용자가 지도에서 식당을 선택하면
해당 식당의 일기 작성 페이지로 전달



일기를 작성하면 선택된 식당 정보와 함께
이미지,
위치의 데이터가 FoodDiary 엔티티에 저장

```
{
  "id": 2,
  "diaryText": "교동짬뽕.. 여기 진짜 서구청 주변에서 고기육수인데 깔끔한 맛이 나게 하는 곳은 처음인듯..  
9500원으로 좀 비싸긴 하지만 양도 많고 공깃밥도 무료라 부족하면 추가로 먹을 수 있음!",
  "createdAt": "2025-03-28T12:24:50.570+00:00",
  "restaurantId": 5,
  "photoPath": "/uploads/restaurant_5/2.jpg",
  "heart": false
}
```

일기 API # 일기 CRUD 기능 구현



교동짬뽕

사진 추가

음식 이름

인천 서구 심곡로 35



내용 입력

저장



먹스팟



식당명: 미분당



메뉴: 양지쌀국수
위치: 인천 서구 청라라일로 85

미분당은 혼자먹기좋은 혼밥집. 고수추가도 무료고 조용하게 한끼를 해결할 수 있다. 근데 조용히하라고 직원들이 유별날 때도 있으니 점바점 조심.





식탁일기



식당명: 미분당



메뉴: 양지쌀국수
위치: 인천 서구 청라라일로 85

미분당은 혼자먹기좋은 혼밥집. 고수추가도 무료고 조용하게 한끼를 해결할 수 있다. 근데 조용히하라고 직원들이 유별날 때도 있으니 점바점 조심.



수정
삭제



식탁일기



식당명: 미분당



메뉴: 양지쌀국수
위치: 인천 서구 청라라일로 85

미분당은 혼자먹기좋은 혼밥집. 고수추가도 무료고 조용하게 한끼를 해결할 수 있다. 근데 조용히하라고 직원들이 유별날 때도 있으니 점바점 조심.



삭제 확인
정말로 이 일기를 삭제하시겠습니까?

취소
삭제



식탁일기

일기 API # 일기 CRUD 기능 구현

POST

http://13.125.46.254:8080/fooddiary

ParamsAuthorizationHeaders (8)Body ●Scripts ●Tests ●Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key	Value
☒	restaurantId	Text 1
☒	diaryText	Text 초밥이 왕맛있다!
☒	photo	File 1.jpg

BodyCookiesHeaders (5)Test Results (1/1)

RawPreviewVisualize

1 일기가 성공적으로 추가되었습니다.

DELETE

http://localhost:8080/fooddiary/7

ParamsAuthorizationHeaders (6)Body ●Scripts ●Tests ●Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key	Value
	Key	Text Value

BodyCookiesHeaders (5)Test Results (1/1)

RawPreviewVisualize

1 일기가 성공적으로 삭제되었습니다.

http://localhost:8080/fooddiary/7

AuthorizationHeaders (8)Body ●Scripts ●Tests ●Settings

form-data

x-www-form-urlencoded

raw

binary

GraphQL

cookiesHeaders (5)Test Results (1/1)

200 OK

PreviewVisualize

일기가 성공적으로 수정되었습니다.

일기 API # 일기장 피드 구현

총 5개



```
@GetMapping("/feed")
public ResponseEntity<List<FoodDiary>> getFeed() {
    // 정렬 조건에 따라
    if (sort.equals("desc")) {
        diaries = diaries.sorted(Comparator.reverseOrder());
    } else {
        // 기본 정렬:
        diaries = diaries;
    }

    // DTO 매핑
    List<FoodDiaryFeed> feed = diaries.map(diary -> new FoodDiaryFeed(
        diary.getId(),
        diary.getPhotoPath(),
        diary.getHeart()
    ));

    return ResponseEntity.ok(feed);
}
```

Body Cookies Headers (5) Test Results (1/1) 200 OK • 986 ms • 547 B • Save Response

JSON Preview Visualize

```
[
  {
    "id": 6,
    "photoPath": "/photos/restaurant_30/6.jpg",
    "heart": true
  },
  {
    "id": 5,
    "photoPath": "/photos/restaurant_20/5.jpg",
    "heart": true
  },
  {
    "id": 4,
    "photoPath": "/photos/restaurant_15/4.jpg",
    "heart": true
  },
  {
    "id": 3,
    "photoPath": "/photos/restaurant_10/3.jpg",
    "heart": true
  },
  {
    "id": 2,
    "photoPath": "/photos/restaurant_5/2.jpg",
    "heart": true
  },
  {
    "id": 1,
    "photoPath": "/photos/restaurant_1/1.jpg",
    "heart": false
  }
]
```

Body Cookies Headers (5) Test Results (1/1) 200 OK • 1.14 s • 548 B • Save Response

JSON Preview Visualize



한입만

Front-end

주요기능개발

#AndroidStudio

#Retrofit2

#Glide



Open API

카카오 로그인 & 마이페이지 API

로그인

If you don't have an account
please Sign up here

아이디

비밀번호

회원이 아니신가요?

로그인

Or



카카오 로그인

APP Hannipman
Hannipman

전체 동의하기

Hannipman 서비스 제공을 위해 회원번호와 함께 개인 정보가 제공됩니다. 보다 자세한 개인정보 제공항목은 동의 내용에서 확인하실 수 있습니다. 해당 정보는 동의 철회 또는 서비스 탈퇴 시 지체없이 파기됩니다.

✓ [필수] 카카오 개인정보 제3자 제공 동의 보기

동의하고 계속하기

취소



인천광역시 서구 불로동 736

내 주변 찐맛집을 찾아보세요!



오늘 뭐 먹을까?



한식



중식



일식



양식

오늘의 추천 메뉴!

wait . . .

추천 식당

★ 0.0

추천 메뉴: -

가게 보기

다른 가게 추천

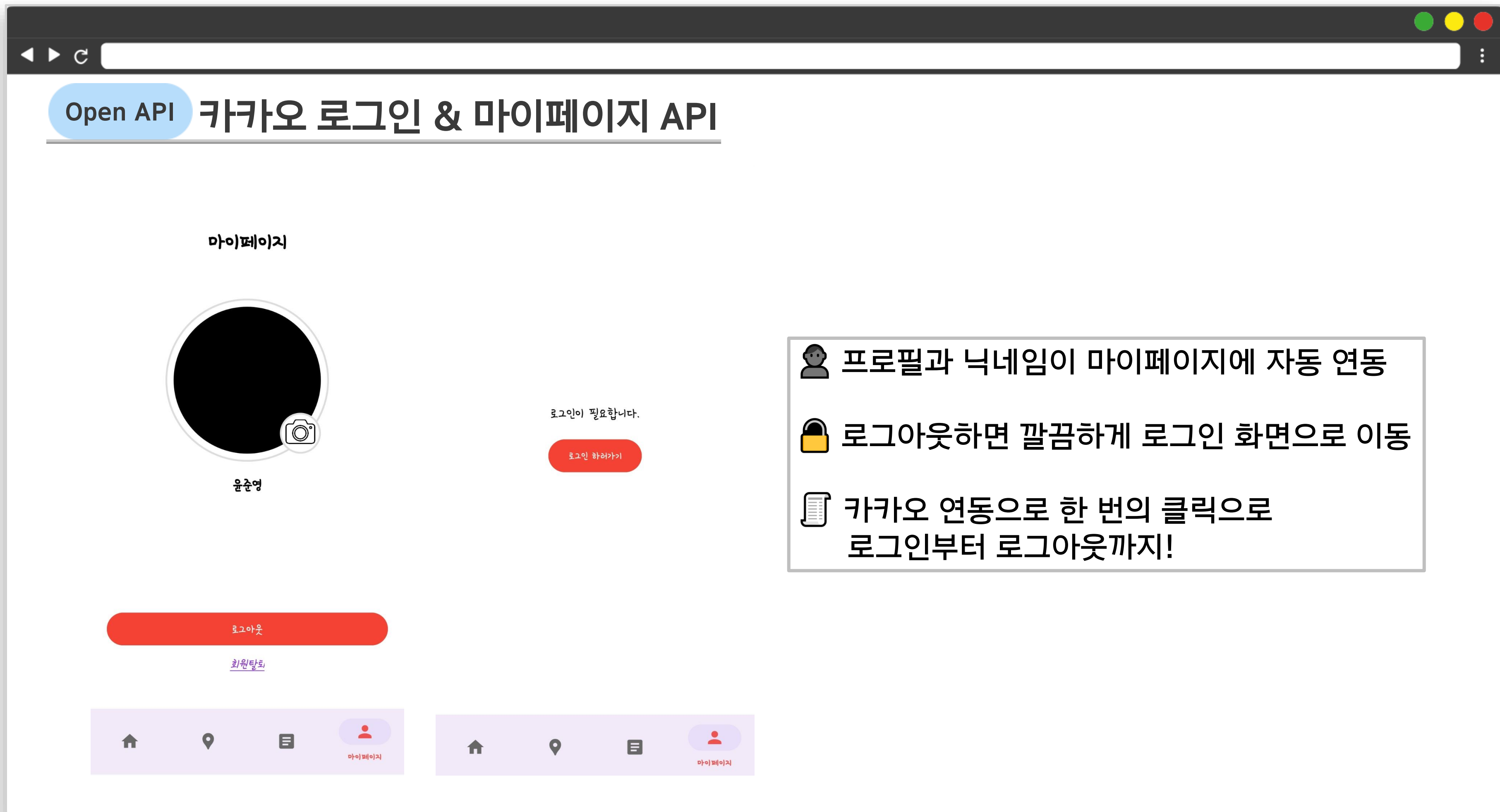


카카오 로그인 성공



홈





Open API 카카오 로그인 & 마이페이지 API

카카오 마이페이지 메서드

```
private void loadUserProfile() {
    UserApiClient.getInstance().me((User user, Throwable throwable) -> {
        if (throwable != null) {
            Log.e(tag: "KakaoLogin", msg: "사용자 정보 요청 실패", throwable);
            Toast.makeText(getActivity(), text: "사용자 정보를 가져오는 데 실패했습니다.", Toast.LENGTH_SHORT).show();
        } else if (user != null && user.getKakaoAccount() != null && user.getKakaoAccount().getProfile() != null) {
            Log.d(tag: "KakaoLogin", msg: "사용자 정보 요청 성공");

            // 닉네임 설정
            String nickname = user.getKakaoAccount().getProfile().getNickname();
            nicknameText.setText(nickname != null ? nickname : "사용자");

            // 프로필 이미지 설정
            String profileUrl = user.getKakaoAccount().getProfile().getProfileImageUrl();
            Glide.with(fragment: this) RequestManager
                .load(profileUrl) RequestBuilder<Drawable>
                .placeholder(R.drawable.user) // 로딩 중 기본 이미지
                .error(R.drawable.user) // 실패 시 기본 이미지
                .circleCrop() // 프로필 이미지를 원형으로 표시!
                .into(profileImage);
        } else {
            Log.e(tag: "KakaoLogin", msg: "프로필 정보가 없습니다.");
            Toast.makeText(getActivity(), text: "프로필 정보를 불러올 수 없습니다.", Toast.LENGTH_SHORT).show();
        }
        return null;
    });
}
```

```
// 로그아웃 버튼 클릭 처리
logoutButton.setOnClickListener(v -> {
    UserApiClient.getInstance().logout(error -> {
        if (error != null) {
            Log.e(TAG, "로그아웃 실패. SDK에서 토큰 폐기됨", error);
        } else {
            Log.i(TAG, "로그아웃 성공. SDK에서 토큰 폐기됨");
            Toast.makeText(getActivity(), "로그아웃 되었습니다.", Toast.LENGTH_SHORT).show();

            Intent intent = new Intent(getActivity(), LoginActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
            intent.putExtra("from", "mypage");
            startActivity(intent);
        }
        return null;
    });
});
```

✓ loadUserProfile() 메서드 사용
-> 카카오 프로필(닉네임 + 이미지) 표시

✓ Glide로 원형 이미지 처리

✓ 로그아웃 버튼 → 토큰 폐기 + 로그인 화면 이동

메인화면 구조

홈



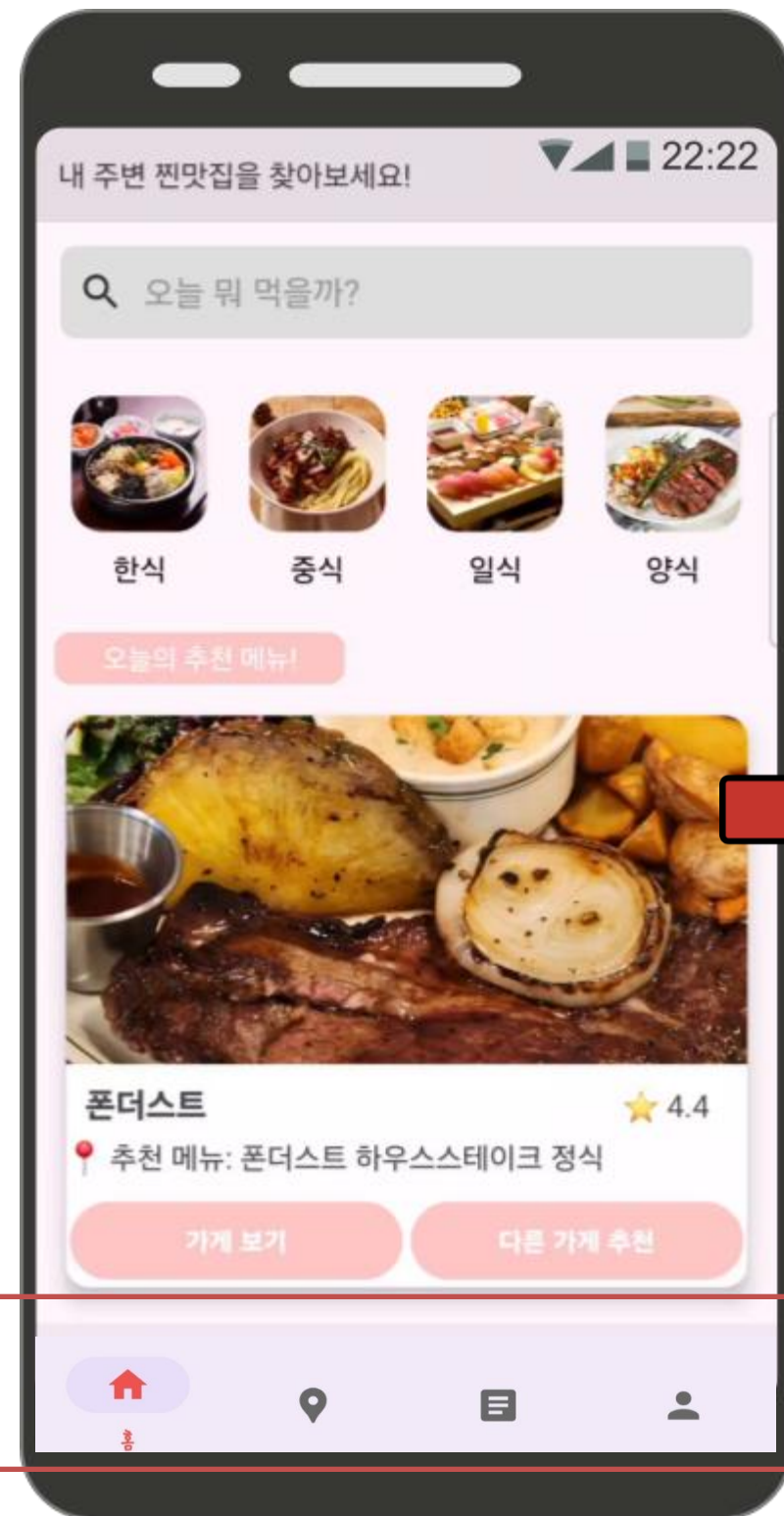
일기



지도



마이페이지



1. 식당 검색

2. 카테고리 분류

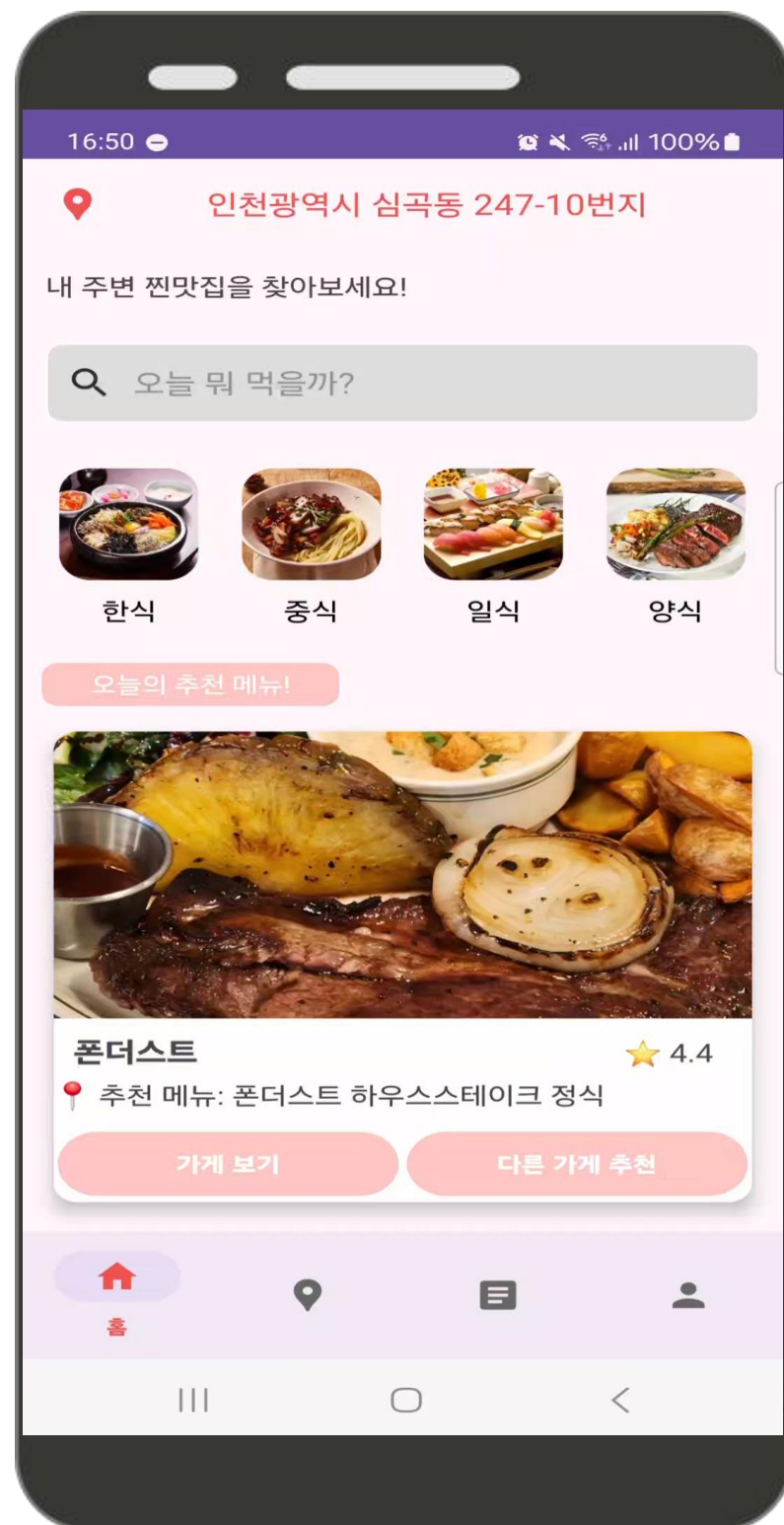
3. 랜덤 가게 추천

4. 광고 배너

메인 홈

식당 카테고리 분류 및 상세정보

- 식당 클릭 시 상세 페이지로 이동
- fetchAllRestaurants()를 통해 모든 식당 정보를 가져온 후, 한식, 일식, 중식, 분식 등 카테고리별로 필터링하여 리스트로 출력
- 식당 이름, 번호, 운영 시간, 휴무일, 위치, 평점, 키오스크 유무, 메뉴판 등 표시
- Glide를 활용해 식당 이미지를 서버에서 실시간 로드
- 사용자 경험을 향상시키기 위해 UI를 직관적으로 구성



검색 / 랜덤 식당 추천 / 광고

- 사용자가 식당 이름 입력하여 원하는 식당을 빠르게 검색
- 홈 화면에 상단에 랜덤 추천 기능을 배치하여 사용자 경험(UX)을 최우선으로 고려
- showRandomRecommendation()을 통해 모든 식당 리스트에서 무작위로 한 곳을 선택하여 추천
- 추천된 식당은 버튼 클릭 시 갱신되며, '가게 보기' 버튼을 누르면 상세 페이지로 이동

메인 홈

RetrofitClient.java

```
public class RetrofitClient {
    3 usages
    private static Retrofit retrofit = null;

    4 usages
    public static Retrofit getInstance() { // 기존 getRetrofitInstance() 유지
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl("http://13.125.46.254:8080") // 백엔드 서버 주소
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit;
    }

    /**
     * ☒ DiaryApiService 객체 반환 (API 요청을 쉽게 처리)
     */
    5 usages
    public static DiaryApiService getDiaryApiService() {
        return getInstance().create(DiaryApiService.class);
    }

    5 usages
    public static RestaurantApiService getRestaurantApiService() {
        return getInstance().create(RestaurantApiService.class);
    }
}
```

RestaurantApiService.java

```
public interface RestaurantApiService {

    // 주변 식당 리스트 가져오기
    4 usages
    @GET("/restaurants/json")
    Call<List<Restaurant>> getAllRestaurants();

    // 식당 상세 정보 가져오기
    1 usage
    @GET("/restaurants/json/{id}") // 경로 수정
    Call<Restaurant> getRestaurantDetail(@Path("id") int id);
}
```

- Retrofit을 초기화, Base URL을 설정하여 서버와의 통신을 처리
모든 API 요청을 처리하는 Retrofit 인스턴스를 생성하고 관리
- 전체 식당 리스트와 특정 식당의 상세 정보 API를 정의

메인 홈

RestaurantList.java

```
Call<List<Restaurant>> call = restaurantApi.getAllRestaurants();
call.enqueue(new Callback<List<Restaurant>>() {
    @Override
    public void onResponse(Call<List<Restaurant>> call, Response<List<Restaurant>> response) {
        if (response.isSuccessful() && response.body() != null) {
            allRestaurants.clear();
            allRestaurants.addAll(response.body()); // 전체 식당 데이터 저장
            filterRestaurantsByCategory(category); // 필터링하여 UI 업데이트
        } else {
            Toast.makeText(context, RestaurantListActivity.this, "식당 데이터 불러오기 실패", Toast.LENGTH_SHORT).show();
        }
    }

    @Override
    public void onFailure(Call<List<Restaurant>> call, Throwable t) {
        Toast.makeText(context, RestaurantListActivity.this, "네트워크 오류", Toast.LENGTH_SHORT).show();
    }
});
```

1. 식당 ID 전달
2. Retrofit을 통한 비동기 데이터 요청
3. UI 업데이트
4. 메뉴 및 가격 정보
5. 이미지 로딩

RestaurantDetail.java

```
restaurantApi.getRestaurantDetail(id).enqueue(new Callback<Restaurant>() {
    @Override
    public void onResponse(Call<Restaurant> call, Response<Restaurant> response) {
        if (response.isSuccessful() && response.body() != null) {
            Restaurant restaurant = response.body();
            Log.d(tag, "DetailAPI", msg, "식당 이름: " + restaurant.getName());
            Log.d(tag, "DetailAPI", msg, "식당 주소: " + restaurant.getPlace());
            Log.d(tag, "DetailAPI", msg, "식당 전화번호: " + restaurant.getPhoneNumber());
            Log.d(tag, "DetailAPI", msg, "식당 영업시간: " + restaurant.getBusinessHours());
            Log.d(tag, "DetailAPI", msg, "식당 이미지 URL: " + restaurant.getImage());
            Log.d(tag, "DetailAPI", msg, "메뉴 목록: " + restaurant.getMenuList());
            Log.d(tag, "DetailAPI", msg, "가격 목록: " + restaurant.getPriceList());

            updateUI(restaurant);
        } else {
            try {
                String errorBody = response.errorBody().string();
                Log.e(tag, "DetailAPI", msg, "에러 응답 내용: " + errorBody);
            } catch (IOException e) {
                Log.e(tag, "DetailAPI", msg, "에러 바디 읽기 실패", e);
            }
            Toast.makeText(context, RestaurantDetailActivity.this, "데이터를 불러오지 못했습니다.", Toast.LENGTH_SHORT).show();
            clearUI();
        }
    }

    @Override
    public void onFailure(Call<Restaurant> call, Throwable t) {
        Log.e(tag, "DetailAPI", msg, "네트워크 오류 발생: " + t.getMessage());
        Toast.makeText(context, RestaurantDetailActivity.this, "네트워크 오류 발생", Toast.LENGTH_SHORT).show();
    }
});
```

메인 홈

카테고리 or 검색

```
private void filterRestaurantsByCategory(String category) {
    List<Restaurant> filteredList = new ArrayList<>();

    for (Restaurant restaurant : allRestaurants) {
        if (restaurant.getCategory() != null && restaurant.getCategory().trim().equals(category)) {
            filteredList.add(restaurant);
        }
    }

    if (filteredList.isEmpty()) {
        restaurantAdapter.updateList(allRestaurants);
    } else {
        restaurantAdapter.updateList(filteredList);
    }
}
```

```
1 usage
private void filterRestaurantsByQuery(String query) {
    List<Restaurant> filteredList = new ArrayList<>();

    for (Restaurant restaurant : allRestaurants) {
        if (restaurant.getName().toLowerCase().contains(query.toLowerCase())) {
            filteredList.add(restaurant);
        }
    }

    if (filteredList.isEmpty()) {
        restaurantAdapter.updateList(allRestaurants);
    } else {
        restaurantAdapter.updateList(filteredList);
    }
}
```

랜덤 식당 추천

```
private void showRandomRecommendation() {
    if (!restaurantList.isEmpty()) {
        Random random = new Random();
        int index = random.nextInt(restaurantList.size());
        Restaurant randomRestaurant = restaurantList.get(index);

        selectedRestaurant = randomRestaurant;

        recommendedRestaurantName.setText(randomRestaurant.getName());
        recommendedRestaurantMenu.setText("🍴 추천 메뉴: " + (randomRestaurant.getMenuList().isEmpty() ? "-" : randomRestaurant.getMenuList().get(0)));
        recommendedRestaurantRating.setText("★ " + randomRestaurant.getRating());

        if (isAdded() && getContext() != null) {
            Glide.with(requireContext()).requestManager()
                .load(randomRestaurant.getImage())
                .into(recommendedRestaurantImage);
        }
    }

    btnViewRestaurant.setOnClickListener( View v -> {
        if (selectedRestaurant != null) {
            Intent intent = new Intent(getActivity())
                .putExtra("name", "restaurantId", selectedRestaurant.getId())
                .startActivity(intent);
        }
    });
}
```

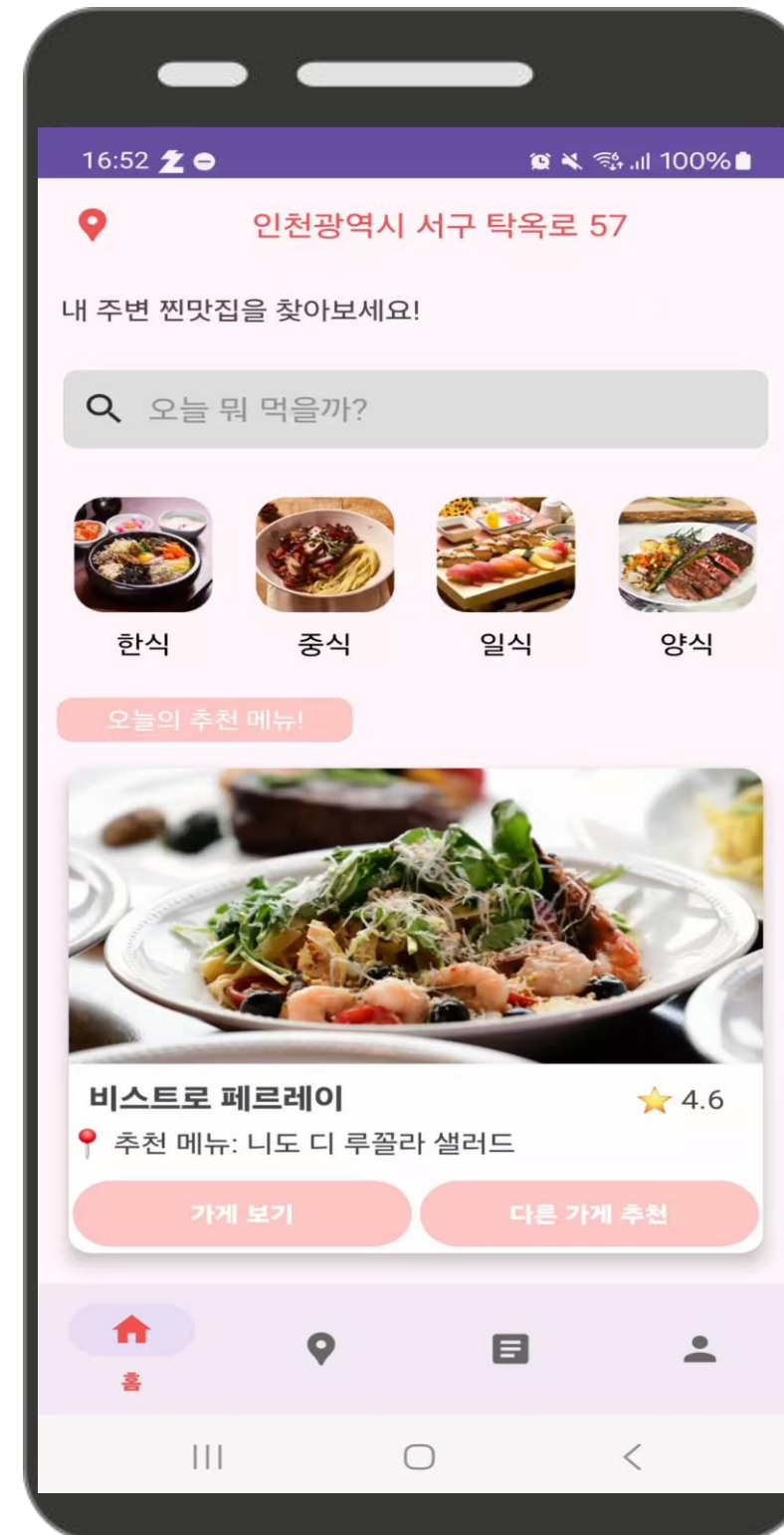
광고 배너

```
// 광고 배너 프래그먼트 (ViewPager 2)
adBannerViewPager = view.findViewById(R.id.ad_banner_viewpager);
adBannerImages = new ArrayList<>();
adBannerImages.add(R.drawable.banner_hamburger);
adBannerImages.add(R.drawable.banner_pizza);
adBannerImages.add(R.drawable.banner_korean);
adBannerAdapter = new AdBannerAdapter(getContext(), adBannerImages);
adBannerViewPager.setAdapter(adBannerAdapter);
adBannerViewPager.setOffscreenPageLimit(3);
adBannerViewPager.setClipToPadding(false);
adBannerViewPager.setClipChildren(false);
adBannerViewPager.setPadding(left: 20, top: 0, right: 160, bottom: 0);
adBannerViewPager.setPageTransformer(( View page, float position) -> {
    float scaleFactor = 0.9f + (1 - Math.abs(position)) * 0.1f;
    page.setScaleY(scaleFactor); // 배너 크기 조정
    page.setAlpha(0.8f + (1 - Math.abs(position)) * 0.2f); // 투명도 조정
});
```


지도(먹스팟)

1. 구글 맵 기반 식당 정보를 받아 현재 위치 기준 주변 위도, 경도 기준 마커표시
2. 클릭리스너로 바텀시트를 불러오고 해당 가게 정보를 서버에 요청
3. 번들로 바텀시트의 일기 작성 버튼 -> 에딧 페이지 이동

```
private void fetchRestaurantData() {
    RetrofitClient.getRestaurantApiService().getAllRestaurants().enqueue(new Callback<List<Restaurant>>() {
        @Override
        public void onResponse(Call<List<Restaurant>> call, Response<List<Restaurant>> response) {
            if (response.isSuccessful() && response.body() != null) {
                List<Restaurant> restaurantList = response.body();
                for (Restaurant restaurant : restaurantList) {
                    LatLng position = new LatLng(restaurant.getLatitude(), restaurant.getLongitude());
                    MarkerOptions markerOptions = new MarkerOptions()
                        .position(position)
                        .title(restaurant.getName())
                        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_RED));
                    Marker marker = mMap.addMarker(markerOptions);
                    markerRestaurantMap.put(marker, restaurant);
                }
            }
        }
    });
}
```



```
mMap.setOnMarkerClickListener( Marker marker -> {
    selectedRestaurant = markerRestaurantMap.get(marker);
    if (selectedRestaurant != null) {
        showBottomSheet(selectedRestaurant);
        fetchDiaryDataForRestaurant(selectedRestaurant.getId());
    }
    return false;
});
});
```

```
3 usages
private void uploadDiaryToServer(MultipartBody.Part photoPart, RequestBody restaurantId,
    RequestBody diaryText, RequestBody rating, RequestBody menuName) {
    Log.d(TAG, msg: "새 일기 작성 요청:");
    Log.d(TAG, msg: "restaurantId = " + bodyToString(restaurantId));
    Log.d(TAG, msg: "diaryText = " + bodyToString(diaryText));
    Log.d(TAG, msg: "rating = " + bodyToString(rating));
    Log.d(TAG, msg: "menuName = " + bodyToString(menuName));
    Log.d(TAG, msg: "photoPart = " + (photoPart != null ? "있음" : "없음"));

    diaryApiService.createDiaryWithImages(restaurantId, diaryText, rating, menuName, photoPart)
        .enqueue(new Callback<ResponseBody>() {
            @Override
            public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
                if (response.isSuccessful()) {
                    Toast.makeText(getContext(), text: "일기가 저장되었습니다!", Toast.LENGTH_SHORT).show();
                    requireActivity().getSupportFragmentManager().popBackStack();
                } else {
                    Log.e(TAG, msg: "저장 실패 - 코드: " + response.code());
                    try {
                        if (response.errorBody() != null) {
                            Log.e(TAG, msg: "에러 내용: " + response.errorBody().string());
                        }
                    } catch (IOException e) {
                        e.printStackTrace();
                    }
                    Toast.makeText(getContext(), text: "저장 실패", Toast.LENGTH_SHORT).show();
                }
            }
            @Override
            public void onFailure(Call<ResponseBody> call, Throwable t) {
                Log.e(TAG, msg: "저장 실패", t);
                Toast.makeText(getContext(), text: "저장 실패: " + t.getMessage(), Toast.LENGTH_SHORT).show();
            }
        });
}
```

일기(식탁일기)

일기 불러오기

```
private void fetchDiariesFromServer() {
    Log.d(tag, "DiaryFetch", msg: "📡 서버에 피드 요청 시작*");

    diaryApiService.getFeedPhotos().enqueue(new Callback<List<FoodDiaryFeedResponse>>() {
        @Override
        public void onResponse(Call<List<FoodDiaryFeedResponse>> call, Response<List<FoodDiaryFeedResponse>> response) {
            if (response.isSuccessful() && response.body() != null) {
                diaryEntries.clear();
                diaryEntries.addAll(response.body());

                Log.d(tag, "DiaryFetch", msg: "✅ 피드 응답 성공 - 개수: " + diaryEntries.size());
                for (FoodDiaryFeedResponse response : response.body()) {
                    private void uploadUpdatedDiaryToServer(MultipartBody.Part photoPart, RequestBody restaurantId,
                        RequestBody diaryText, RequestBody rating, RequestBody menuName) {
                        Log.d(TAG, msg: "📝 일기 수정 요청 - diaryId: " + diaryId);
                        Log.d(TAG, msg: "📍 restaurantId = " + bodyToString(restaurantId));
                        Log.d(TAG, msg: "📝 diaryText = " + bodyToString(diaryText));
                        Log.d(TAG, msg: "📊 rating = " + bodyToString(rating));
                        Log.d(TAG, msg: "🍽️ menuName = " + bodyToString(menuName));
                        Log.d(TAG, msg: "📷 photoPart = " + (photoPart != null ? "있음" : "없음"));

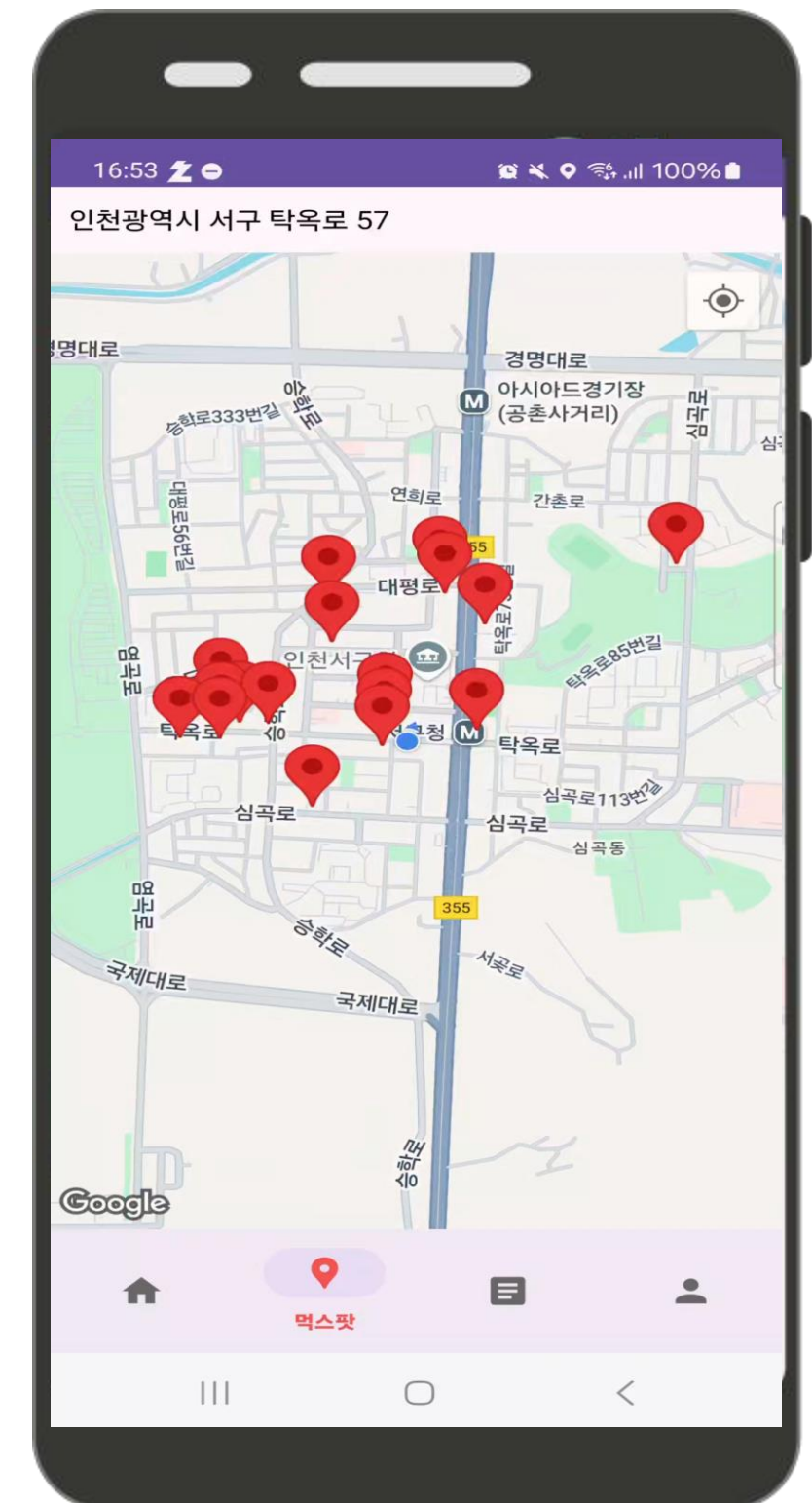
                        diaryApiService.updateDiaryWithImages(diaryId, restaurantId, diaryText, rating, menuName, photoPart)
                            .enqueue(new Callback<ResponseBody>() {
                                @Override
                                public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
                                    if (response.isSuccessful()) {
                                        Toast.makeText(getApplicationContext(), "일기 수정 성공", Toast.LENGTH_SHORT).show();
                                    } else {
                                        Log.e(TAG, msg: "❌ 수정 실패 코드: " + response.code());
                                        Toast.makeText(getApplicationContext(), "일기 수정 실패", Toast.LENGTH_SHORT).show();
                                    }
                                }
                            });
                    }
                }
            } else {
                Log.e(tag, "DiaryFetch", msg: "❌ 서버 응답 실패");
            }
        }
    });
}
```

일기 수정하기

일기 삭제하기

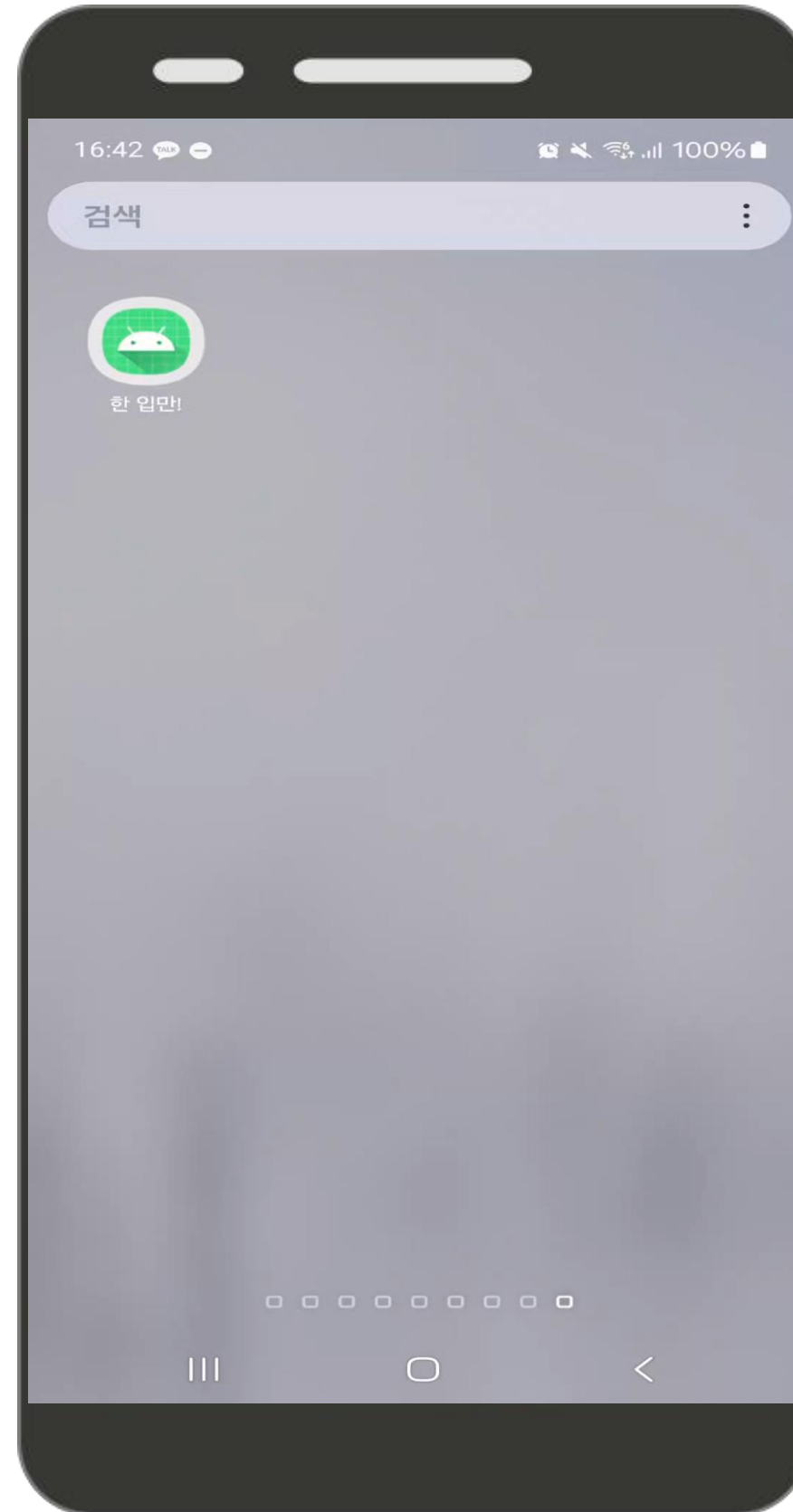
```
private void deleteDiary() {
    if (diaryId == null || diaryId == -1) return;

    diaryApiService.deleteDiaryWithImages(diaryId).enqueue(new Callback<ResponseBody>() {
        @Override
        public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
            if (response.isSuccessful()) {
                Log.d(TAG, msg: "🗑️ 일기 삭제 성공*");
                Toast.makeText(getApplicationContext(), "일기가 삭제되었습니다", Toast.LENGTH_SHORT).show();
                requireActivity().getSupportFragmentManager().popBackStack();
            } else {
                Log.e(TAG, msg: "❌ 삭제 실패 코드: " + response.code());
                Toast.makeText(getApplicationContext(), "삭제 실패", Toast.LENGTH_SHORT).show();
            }
        }
    });
}
```





시연 영상



프로젝트 진행 중 발생한 문제, 해결방법

Err

멀티파트 파일 처리 방식과 EC2 서버 파일
경로 문제로 이미지 파일이 저장 안됨



Fix

저장 방식 수정과 전용 폴더 생성으로 해결

Err

SDK 버전문제와 키 해시 불일치로
카카오 로그인 자체가 안됨



Fix

최신 SDK 적용 및 키 해시 등록으로
정상 작동하도록 해결

Err

자주 묻는 질문을 입력해주세요.



Fix

질문에 대한 답변을 입력해주세요.

Err

자주 묻는 질문을 입력해주세요.



Fix

질문에 대한 답변을 입력해주세요.



감사합니다
